visualize_main_e2e

February 12, 2026

1 Figure 11. End-to-end performance with Cluster-A.

2 End-to-End Main Figure Visualization

This notebook generates the main end-to-end performance figure, comparing JCT, Weighted JCT, and Utilization across multiple scheduling policies for `clusterA`.

2.1 1. Setup and Configuration

This section defines the configuration, loads data for all policies, and calculates the necessary metrics (JCT, W.JCT, Utilization). An event-driven analysis function will be used for accuracy.

[1]:
```
import torch
import os
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
from collections import defaultdict
from Suika import analyze_job_statistics, parse_cluster_config

# --- Configuration ---
cluster = "clusterA"
policies = ["suika", "sia", "rubick", "dp", "metis", "fcfs"]
config_path = "../configs/cluster64.json"
total_n_acc = parse_cluster_config(config_path)[0].get_total_acc_num()

policy_map = {
    "suika": "Suika",
    "sia": "Sia",
    "rubick": "Rubick",
    "dp": "Suika-DP",
    "metis": "Suika-Metis",
    "fcfs": "FIFO",
}

policy_styles = {
    "suika":  {"color": "#34495e", "hatch": ""},
    "sia":    {"color": "#e67e22", "hatch": "/"},
```

```
    "rubick": {"color": "#f1c40f", "hatch": "x"},
    "dp":  {"color": "#95a5a6", "hatch": "+"},
    "metis":     {"color": "#3498db", "hatch": "."},
    "fcfs":   {"color": "#e74c3c", "hatch": "-"},
}

all_stats = defaultdict(dict)

print(f"Cluster: {cluster} ({total_n_acc} GPUs)")
```

```
/home/fake/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Cluster: clusterA (64 GPUs)
```

2.2 2. Event-Driven Analysis Function

[2]:
```
from collections import defaultdict
import numpy as np

def analyze_and_extract_data_event_driven(stats, total_n_acc):
    events = []
    for job_id, job_data in stats.items():
        # Submit Event
        events.append((job_data['submit_time'], 'submit', job_id, 0))

        # Start, Reconfig, and End Events
        for i, start_time in enumerate(job_data['reconfig_start_time']):
            if i >= len(job_data['reconfig_stop_time']):
                break
            n_acc = job_data['n_acc'][i]
            old_acc = job_data['n_acc'][i-1] if i > 0 else 0
            if n_acc == 0:
                events.append((start_time, 'preempt', job_id, 0))
            elif old_acc == 0:
                events.append((start_time, 'start', job_id, n_acc))
            else:
                events.append((start_time, 'reconfig', job_id, n_acc))

        if job_data['end_time']:
            events.append((job_data['end_time'][-1], 'end', job_id, 0))

    events.sort(key=lambda x: x[0])

    timeline = []
    n_acc_current = 0
```

```
    running_jobs_count = 0
    waiting_jobs_count = 0
    job_n_acc = defaultdict(int)
    total_gpu_time = 0
    last_event_time = 0

    timeline.append({
        'time': 0, 'n_acc': 0, 'running': 0, 'waiting': 0,
        'submit': 0, 'start': 0, 'end': 0
    })

    for event in events:
        time, event_type = event[0], event[1]
        time_delta = time - last_event_time
        if time_delta > 0:
            total_gpu_time += n_acc_current * time_delta

        new_point = timeline[-1].copy()
        new_point.update({'time': time, 'submit': 0, 'start': 0, 'end': 0})

        if event_type == 'submit':
            waiting_jobs_count += 1
            new_point['submit'] = 1
        elif event_type == 'start':
            job_id, n_acc = event[2], event[3]
            waiting_jobs_count -= 1
            running_jobs_count += 1
            n_acc_current += n_acc
            job_n_acc[job_id] = n_acc
            new_point['start'] = 1
        elif event_type == 'end':
            job_id = event[2]
            running_jobs_count -= 1
            n_acc_current -= job_n_acc[job_id]
            del job_n_acc[job_id]
            new_point['end'] = 1
        elif event_type == 'reconfig':
            job_id, new_n_acc = event[2], event[3]
            old_n_acc = job_n_acc[job_id]
            n_acc_current += (new_n_acc - old_n_acc)
            job_n_acc[job_id] = new_n_acc
        elif event_type == 'preempt':
            job_id = event[2]
            n_acc_to_remove = job_n_acc[job_id]
            n_acc_current -= n_acc_to_remove
            job_n_acc[job_id] = 0
            running_jobs_count -= 1
```

```
            waiting_jobs_count += 1

        new_point.update({
            'n_acc': n_acc_current,
            'running': running_jobs_count,
            'waiting': waiting_jobs_count
        })

        if new_point['time'] == timeline[-1]['time']:
            for key in ['submit', 'start', 'end', 'running', 'waiting',␣
↪'n_acc']:
                if key in ['submit', 'start', 'end']:
                    timeline[-1][key] += new_point[key]
                else:
                    timeline[-1][key] = new_point[key]
        else:
            timeline.append(new_point)

        last_event_time = time

    make_span = events[-1][0] if events else 0
    analyzed_stats_base = analyze_job_statistics(stats)
    cluster_utilization = (total_gpu_time / (make_span * total_n_acc)) * 100 if␣
↪make_span > 0 else 0

    final_stats = analyzed_stats_base
    final_stats.pop('per_job_statistics', None)
    final_stats.pop('job_status', None)
    final_stats["cluster_utilization"] = cluster_utilization

    return timeline, final_stats
```

2.3 3. Data Loading and Processing

Now, we load the data for each policy and process it using the event-driven function defined above.

[3]:
```
print("Loading data...")
for policy in policies:
    file_path = f"../results/{cluster}/{policy}-{cluster}.pth"
    try:
        stats = torch.load(file_path, weights_only=False)
        _, analyzed = analyze_and_extract_data_event_driven(stats, total_n_acc)

        avg_jct = analyzed.get("avg_jct")
        avg_w_jct = analyzed.get("avg_w_jct")
        utilization = analyzed.get("cluster_utilization")
```

```
        if avg_jct is not None and avg_w_jct is not None and utilization is not␣
 ↪None:
            all_stats[policy] = {
                'jct': avg_jct,
                'w_jct': avg_w_jct,
                'util': utilization,
            }
            print(f"  - Loaded {policy}: JCT={avg_jct:.2f}, W.JCT={avg_w_jct:.
 ↪2f}, Util={utilization:.2f}%")
        else:
            print(f"  - WARNING: Could not find all required data for {policy}")

    except FileNotFoundError:
        print(f"  - SKIPPING: File not found at {file_path}")
    except Exception as e:
        print(f"  - ERROR: Failed to load or process {file_path}: {e}")
        raise e
print("\nData loading complete.")
```

```
Loading data…
  - Loaded suika: JCT=2127.41, W.JCT=3531.14, Util=96.17%
  - Loaded sia: JCT=2735.18, W.JCT=4825.25, Util=90.85%
  - Loaded rubick: JCT=2786.77, W.JCT=4746.97, Util=94.08%
  - Loaded dp: JCT=2517.64, W.JCT=4194.92, Util=86.55%
  - Loaded metis: JCT=2888.06, W.JCT=4906.40, Util=73.03%
  - Loaded fcfs: JCT=3561.93, W.JCT=5185.36, Util=60.31%

Data loading complete.
```

2.4 4. Visualization

This section generates the main figure, which is composed of two subplots separated by a vertical line: - **Left Plot**: Grouped bar charts for Average JCT and Average Weighted JCT. - **Right Plot**: A bar chart for Cluster Utilization.

A shared legend is placed at the top.

[4]:
```
def plot_main_figure(stats, policy_map, policy_styles):
    metrics_to_plot = ['jct', 'w_jct', 'util']

    valid_policies = [p for p in policies if all(m in stats.get(p, {}) for m in␣
 ↪metrics_to_plot)]
    if not valid_policies:
        print("No policies with complete data to plot.")
        return

    fig = plt.figure(figsize=(14, 5))
    gs = GridSpec(1, 2, width_ratios=[2.3, 1], wspace=0.02)
```

```
    # --- Left Plot (JCT and W.JCT) ---
    ax1 = fig.add_subplot(gs[0])

    metric_groups = ['jct', 'w_jct']
    n_metrics = len(metric_groups)
    n_policies = len(valid_policies)

    x = np.arange(n_metrics)
    width = 0.8 / n_policies

    for i, policy in enumerate(valid_policies):
        metric_values = [stats[policy][m] for m in metric_groups]
        style = policy_styles.get(policy, {})
        offset = (i - (n_policies - 1) / 2) * width

        bars = ax1.bar(x + offset, metric_values, width,
                       label=policy_map.get(policy, policy.upper()),
                       color=style.get("color"), hatch=style.get("hatch"),␣
↪edgecolor='white')

        # Add relative performance markers
        for metric_idx, metric_name in enumerate(metric_groups):
            all_metric_values = [stats[p][metric_name] for p in valid_policies]
            min_metric = min(all_metric_values)
            bar = bars[metric_idx]
            height = bar.get_height()
            relative_perf = height / min_metric
            ax1.annotate(f'{relative_perf:.2f}x',
                         xy=(bar.get_x() + bar.get_width() / 2, height),
                         xytext=(0, 3),  # 3 points vertical offset
                         textcoords="offset points",
                         ha='center', va='bottom', fontsize=12)


    ax1.set_ylabel('Time (s)', fontsize=16, fontweight='bold')
    ax1.set_xticks(x)
    ax1.set_xticklabels(['(a) JCT', '(b) W.JCT'], fontsize=16,␣
↪fontweight='bold')
    ax1.tick_params(axis='y', labelsize=12)
    ax1.grid(True, axis='y', linestyle='--', alpha=0.7)

    # Adjust Y-axis limit for more margin
    current_ylim = ax1.get_ylim()
    ax1.set_ylim(current_ylim[0], current_ylim[1] * 1.2)

    # --- Right Plot (Utilization) ---
```

```
    ax2 = fig.add_subplot(gs[1])

    util_values = [stats[p]['util'] for p in valid_policies]
    x_util = np.arange(1) # Single group for utilization

    # Use the same width as left subplot to ensure consistent bar widths
    for i, policy in enumerate(valid_policies):
        style = policy_styles.get(policy, {})
        offset = (i - (n_policies - 1) / 2) * width
        bar = ax2.bar(x_util + offset, util_values[i], width=width,
                      color=style.get("color"), hatch=style.get("hatch"),␣
↪edgecolor='white')

        # Add concrete value markers
        height = bar[0].get_height()
        ax2.annotate(f'{height:.1f}%',
                     xy=(bar[0].get_x() + bar[0].get_width() / 2, height),
                     xytext=(0, 3),  # 3 points vertical offset
                     textcoords="offset points",
                     ha='center', va='bottom', fontsize=12)


    ax2.set_ylabel('Utilization (%)', fontsize=16, fontweight='bold')
    ax2.yaxis.set_label_position("right")
    ax2.yaxis.tick_right()
    ax2.set_xticks([])
    ax2.tick_params(axis='y', labelsize=12)
    ax2.grid(True, axis='y', linestyle='--', alpha=0.7)
    ax2.set_xlabel("(c) Utilization", fontsize=16, fontweight='bold')

    ax2.set_ylim(0, 105)
    # Add a dotted line at 100% to indicate theoretical maximum
    ax2.axhline(y=100, color='red', linestyle=':', alpha=0.7, linewidth=1)

    # --- Figure-level Adjustments ---
    # line = plt.Line2D([0.67, 0.67], [0.1, 0.9], transform=fig.transFigure,␣
↪color='gray', linestyle='--', linewidth=2)
    # fig.add_artist(line)

    handles, labels = ax1.get_legend_handles_labels()
    fig.legend(handles, labels, loc='upper center', bbox_to_anchor=(0.5, 1.02),
               ncol=len(valid_policies), fontsize=16, frameon=False)

    fig.tight_layout(rect=[0, 0, 1, 0.92], pad=1.0)

    fig_dir = "../fig"
    os.makedirs(fig_dir, exist_ok=True)
```

```
    filename = f"{fig_dir}/main_e2e_comparison.pdf"
    plt.savefig(filename, dpi=600, bbox_inches="tight")
    print(f"Plot saved as '{filename}'")

    plt.show()

# --- Generate Plot ---
plot_main_figure(all_stats, policy_map, policy_styles)
```

```
/tmp/ipykernel_199848/2257125640.py:97: UserWarning: This figure includes Axes
that are not compatible with tight_layout, so results might be incorrect.
  fig.tight_layout(rect=[0, 0, 1, 0.92], pad=1.0)
```

```
Plot saved as '../fig/main_e2e_comparison.pdf'
```

